CSA0824-PYTHON PROGRAMMING

P.YASWANTH REDDY **(192565039)**

Gitup link: https://github.com/yaswanth231107-sudo/CSA0824.githttps://github.com/yaswanth231107-sudo/CSA0824.git

Task 1 – Understand the Problem and Identify the Data

Inputs and outputs

Value	Kind	Type	Why
Farmer ID	Input	str	It is a label (e.g. "F101"); we never do arithmetic on it. Text keeps leading zeros.
Litres	Input	float	Milk is measured in fractions, e.g. 8.5 L.
Fat %	Input	float	Reading such as 4.2.
SNF %	Input	float	Reading such as 8.7.
Shift	Input	str	"M" (morning) or "E" (evening).
Rate per litre	Output	float	Computed from fat and SNF.
Amount	Output	float (2 decimals)	litres × rate, rounded to paise.
Ten-day total	Output	float (2 decimals)	Sum of amounts of one farmer.
Number of entries / steps	Computed	int	Counting 0.1% steps is a whole number.

Named constants (never change during a run)

- BASE_RATE = 30.00 – price for standard milk (3.5% fat, 8.5% SNF).
- FAT_STEP_VALUE = 0.40 and SNF_STEP_VALUE = 0.30 – value of each 0.1% step.
- STD_FAT = 3.5, STD_SNF = 8.5, MIN_FAT = 2.0, MAX_FAT = 7.0 – reference and limits.

They are constants because they are co-operative policy, not farmer data; naming them lets us change a rate in one place.

Whole numbers, fractions, text

- Whole numbers (int): number of steps, number of entries, days (10).
- Fractional (float): litres, fat, snf, rates, amounts.
- Text (str): farmer ID, shift, and all raw keyboard input before conversion.

Rules every payment must satisfy

- Litres must be greater than 0 (no negative or zero quantity).
- Fat must be between 2.0 and 7.0 (inclusive) – outside this the reading is a machine/typing error.
- SNF must be a positive number in a sensible range (assumed 4.0 – 12.0).
- Shift must be M or E. Farmer ID must not be empty. Amount is never negative and is rounded to 2 decimals.

Assumptions

- Fat and SNF are given to one decimal place (0.1 steps); the rate changes in whole steps.
- Standard milk is 3.5% fat and 8.5% SNF; one farmer's ten-day total is the sum of his entries.

Alternative concept and complexity: With three variables per entry (litres1, fat1, snf1, litres2, …) 300 farmers need 900 separate variables and the same formula must be typed 300 times, because a loop cannot step through variable names. Adding a farmer means editing the code. Time is still O(n) and space O(n) in the best case, but the code size is O(n) too and cannot be written for an unknown n. A list of tuples [(litres, fat, snf), …] holds any n entries and one loop (for l, f, s in entries) handles all of them, so the code size is O(1) while only the data grows. That is why the list/tuple design scales and the separate-variable design does not.

Task 2 – Explore Values and Types in the Interpreter

Interactive session (real output)

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> -7 % 2
1
>>> 0.1 + 0.2
```

```
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> Decimal("0.1") + Decimal("0.2") == Decimal("0.3")
True
>>> (4.2 - 3.5) / 0.1
7.000000000000002
>>> round((4.2 - 3.5) / 0.1)
7
>>> int((4.2 - 3.5) / 0.1)
7
>>> math.floor((4.2 - 3.5) / 0.1)
7
>>> (3.1 - 3.5) / 0.1
-3.999999999999999
>>> round(-4.0000000000000036)
-4
>>> int(-3.7), math.floor(-3.7), round(-3.7)
-3, -4, -4
>>> -2 ** 2
-4
>>> (-2) ** 2
4
>>> 2 ** 3 ** 2
512
>>> 2 <= 4.2 <= 7.0
True
>>> round(2.5), round(3.5)
2, 4
>>> round(0.125, 2)
0.12
>>> sys.getsizeof(10), sys.getsizeof(10.0), sys.getsizeof(Decimal("33.40"))
28, 24, 120
>>> 1 + 1 == 2 and not 0 > 1
True
>>> "10" + "10"
'1010'
>>> 10 + 10
20
```

Explanation of the surprising results

- 7 / 2 = 3.5 (true division always gives a float); 7 // 2 = 3 (floor division); 7 % 2 = 1 (remainder).
- -7 // 2 = -4, not -3: floor division rounds down towards minus infinity, and -7 % 2 = 1 keeps the rule a = (a//b)*b + a%b.
- 0.1 + 0.2 = 0.30000000000000004, so == 0.3 is False. Computers store floats in binary and 0.1, 0.2 and 0.3 cannot be stored exactly (like 1/3 in decimal). Decimal stores decimal digits exactly, so it gives True.
- (4.2 - 3.5) / 0.1 gives 7.000000000000002 (the worksheet says ...001; the exact digits depend on the machine but the idea is identical): it is not exactly 7.
- round(2.5) = 2 and round(3.5) = 4: Python rounds exact halves to the nearest even number (banker’s rounding). Also round(0.125, 2) = 0.12.

- -2 ** 2 = -4 because ** binds tighter than unary minus; 2 ** 3 ** 2 = 512 because ** is right-associative.

Why money should not be float rupees

Tiny binary errors add up. Fixes: (1) integer paise (3340 instead of 33.40), (2) round() after every step, (3) the decimal module.

TODO 1 – completed code

```
from decimal import Decimal
payments = ["33.40"] * 1000 + ["0.10"] * 3
def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total
def total_decimal(items):
    total = Decimal("0")
    for amount in items:
        total += Decimal(amount)      # exact: built from the string
    return total
print("float :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3, Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Output:

```
float : 33400.30000000061
Decimal: 33400.30
False True
```

Result: When Decimal(amount) is changed to Decimal(float(amount)) the output becomes 33400.2999999999857893118278. The float 33.40 is already slightly wrong before it reaches Decimal, so Decimal then stores that wrong value exactly. Lesson: build Decimal from a string, never from a float.

Alternative concept and complexity: Measured on 1,003 payments:

Method	Result	Time per addition	Memory per value	Exact?
Decimal (strings)	33400.30	≈ 190 ns	120 bytes	Yes
Integer paise (int)	3340030 paise = 33400.30	≈ 370 ns incl. conversion; plain int add is the fastest	28 bytes	Yes (whole numbers are exact)

float + round() after each add	33400.3	≈ 310 ns	24 bytes	Correct here, but round() can still hit binary half-cases, so not guaranteed

All three are O(1) per addition and O(n) for n payments. Integer paise is smallest and exact; Decimal is exact and easiest to read but uses ~5× the memory of a float.

Task 3 – Choose Variables and Name the Constants

```
BASE_RATE = 30.00       # Rs per litre, standard milk
FAT_STEP_VALUE = 0.40    # Rs per 0.1 % fat
SNF_STEP_VALUE = 0.30    # Rs per 0.1 % SNF
STD_FAT = 3.5
STD_SNF = 8.5
MIN_FAT, MAX_FAT = 2.0, 7.0
```

- Naming rules: letters, digits, underscore; cannot start with a digit; no spaces or keywords; case-sensitive. Convention: snake_case for variables and functions, UPPER_CASE for constants, meaningful names (litres, not x).
- Python has no true constants: any name can be reassigned (BASE_RATE = 5 works without error). UPPER_CASE is only a promise among programmers that the value is not to be changed.
- Dynamic typing: a name just points to an object. x = 10 then x = "ten" is legal; type(x) changes from int to str. This is flexible but x + 1 would then fail with TypeError, so we keep one type per name.
- Multiple assignment: a, b = 1, 2; swapping with a, b = b, a needs no temporary variable.

Alternative concept and complexity: With magic numbers (30.0, 0.40, 0.30 typed inside every expression), a rate revision must be made in k places and any missed one gives wrong payments silently. With named constants, it is one place. With a dictionary RATE_CARD = {"base": 30.0, "fat": 0.40, "snf": 0.30}, it is also one place, and the whole card can be passed to a function (good for seasonal cards); the cost is a lookup RATE_CARD["fat"] and typos in keys appear only at run time. Running time and memory change only slightly (a dictionary lookup is O(1), a few hundred bytes) – practically no difference.

Task 4 – Write Correct Expressions

```
fat_steps = round((fat - STD_FAT) / 0.1)
snf_steps = round((snf - STD_SNF) / 0.1)
rate = round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)
amount = round(litres * rate, 2)
valid = 2.0 <= fat <= 7.0 and litres > 0
```

- Precedence (high to low): ** , unary minus, * / // %, + -, comparisons, not, and, or. In the rate expression the multiplications happen first, then the additions left to right (left-associative). **

is right-associative.

- Comparison chaining: 2.0 <= fat <= 7.0 means 2.0 <= fat and fat <= 7.0 – it reads like mathematics.

Three (or more) surprising results (real output)

Expression	Result	Explanation
(4.2 - 3.5) / 0.1	7.000000000000002	Binary float error; looks like 7 but is not.
(3.1 - 3.5) / 0.1	-3.999999999999999	Almost -4; int() would give -3.
(3.8 - 3.5) / 0.1	2.9999999999999982	int() gives 2 instead of 3 – a step is lost.
-2 ** 2	-4	** before unary minus.
round(2.5)	2	Round-half-even.

round(), int() and math.floor() for negative values

int(-3.7) = -3 (cuts towards zero), math.floor(-3.7) = -4 (down), round(-3.7) = -4 (nearest). round() is the only one that handles the tiny errors above correctly, so it is used for steps.

Alternative concept and complexity: Rate for fat 4.2 and fat 3.1 (SNF 8.7 and 8.3):

Method	fat 4.2 / snf 8.7	fat 3.1 / snf 8.3
round()	33.40	27.80 (correct)
int()	33.10	28.50
math.floor()	33.10	27.80

int() and floor() lose a step whenever the float is just below a whole number (SNF 8.7 → 1.99999 → 1 step instead of 2). For negative values int() goes the wrong way (-3 instead of -4). All three cost O(1) time and O(1) space; only round() is correct, so we choose it.

Task 5 – Read Input and Convert Types

- input() always returns a string, even when the clerk types 8.5. We must convert: litres = float(input("Litres: ")), steps count = int(...).
- "8,5" and "eight" make float() raise ValueError and stop the program.
- The clerk may also type spaces: float(" 8.5 ") works (spaces are ignored), while "nan" and "1e2" also convert, which we may not want.

```
def parse_number(text):
    text = text.strip()
    if text.replace(".", "", 1).isdigit():   # digits with at most one dot
        return float(text)
    return None

litres = parse_number(input("Litres: "))
if litres is None:
    print("Please type a number like 8.5")
```

Real test results of float() and the string check:

Input	float()	String check
8.5	8.5	valid
" 8.5 "	8.5	valid after strip()
8,5	ValueError	rejected
eight	ValueError	rejected
-3	-3.0 (valid number)	rejected (no minus allowed)

Aligned output using an f-string:

```
print(f"{'Ravi':<8}{12:5d}{283.9:10.2f}")
# Ravi       12    283.90
```

Alternative concept and complexity: try: x = float(text) except ValueError: ... is shorter and accepts every valid Python number (" 8.5 ", 1e2, even nan). The string method check is stricter and you decide exactly what is allowed, but it needs more code and can wrongly reject valid forms like "-3". Both take O(length of text) per entry – microseconds – so the time difference is not noticeable. try/except is simpler and clearer; the string check gives better control. "8,5" and "eight" are rejected by both; " 8.5 " is accepted by both once strip() is used.

Task 6 – Organise the Logic into Functions

```
def rate_per_litre(fat, snf):
    """Return price per litre for the given fat and SNF."""
    ...
def payout(litres, fat, snf):
    """Return amount payable for one delivery, rounded to 2 decimals."""
    return round(litres * rate_per_litre(fat, snf), 2)


def ten_day_total(entries):
    """entries is a list of (litres, fat, snf); return the sum of payouts."""
```

```
    total = 0.0
    for litres, fat, snf in entries:
        total += payout(litres, fat, snf)
    return round(total, 2)
```

- Parameters bring data in; the return value gives the answer back to the caller. return gives a value that can be used again (stored, added, tested); print only shows it on screen and the function then returns None.
- Local names exist only inside a function; global names (the rate constants) can be read from any function. Assigning to a constant inside a function would create a new local name (or need the global keyword), which is confusing and dangerous, so constants are only read.
- Every function has a docstring that says what it does and returns.

Alternative concept and complexity: A single long script repeats the rate formula for every place it is needed (more repeated code, higher chance of mistakes, hard to test). A lambda (e.g. rate = lambda f, s: round(30 + ..., 2)) is fine for one-line functions but has no docstring and no statements. I measured 100,000 payments: 0.069 s with functions versus 0.058 s inline – the function-call overhead is about 0.1 µs per call, which is not noticeable. Readability and reuse win.

Task 7 – Build a Module

milkrate.py (TODO 1 and TODO 2 completed)

```
"""Rate card of the village milk co-operative (used as a module)."""
BASE_RATE = 30.00       # Rs per litre for milk with 3.5 % fat and 8.5 % SNF
FAT_STEP_VALUE = 0.40   # Rs per litre for every 0.1 % of fat above or below 3.5
SNF_STEP_VALUE = 0.30   # Rs per litre for every 0.1 % of SNF above or below 8.5
STD_FAT = 3.5
STD_SNF = 8.5

def rate_per_litre(fat, snf):
    """Returns the price per litre, rounded to 2 decimals."""
    fat_steps = round((fat - STD_FAT) / 0.1)
    snf_steps = round((snf - STD_SNF) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    """Returns the amount payable to the farmer, rounded to 2 decimals."""
    return round(litres * rate_per_litre(fat, snf), 2)

if __name__ == "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]:
        rate, pay = rate_per_litre(fat, snf), payout(litres, fat, snf)
        print(f"{litres:5} L fat {fat} snf {snf} rate {rate:6.2f} pay {pay:8.2f}")
```

Run directly (python milkrate.py) – real output:

```
   10 L fat 3.5 snf 8.5 rate  30.00 pay   300.00
  8.5 L fat 4.2 snf 8.7 rate  33.40 pay   283.90
```

```
  12 L fat 3.1 snf 8.3 rate  27.80 pay   333.60
```

Second script (use_milkrate.py):

```
import milkrate
from milkrate import payout
import milkrate as mr
print(milkrate.payout(10, 3.5, 8.5), payout(8.5, 4.2, 8.7), mr.rate_per_litre(3.1, 8.3))

# prints: 300.0 283.9 27.8   (the table is NOT printed)
```

- import milkrate – use names with the prefix: milkrate.payout(...). from milkrate import payout – use payout(...) directly. import milkrate as mr – short alias: mr.payout(...).
- if __name__ == "__main__": the variable __name__ is "__main__" only when the file is run directly; when imported it is "milkrate". So the demo table prints when we run milkrate.py but not when another script imports it. Without this test, importing would print the table every time.
- Standard modules: math (floor, ceil) and decimal (exact money) are imported the same way.
- Python looks for a module in the current folder, then the folders listed in sys.path (standard library, site-packages).

Alternative concept and complexity: Copying the rate functions into every script means a rate change must be made in every copy (k files) and copies can disagree. With a module it is one edit. Importing costs a small one-time load; Python then keeps the module in sys.modules, so importing it again in the same run is just a dictionary lookup.

Task 8 – Plan the Algorithm Before Coding

Steps: (1) Understand – inputs, outputs, rules (Task 1). (2) Plan – pseudocode and flowchart. (3) Write – the Python code. (4) Test – with normal, boundary and invalid cases.

Pseudocode

```
ALGORITHM TenDayPayment
BEGIN
  SET total <- 0
  FOR day <- 1 TO 10 DO
    READ litres, fat, snf
    IF litres <= 0 OR fat < 2.0 OR fat > 7.0 THEN
      DISPLAY "Invalid entry"
    ELSE
      SET fatSteps <- ROUND((fat - 3.5) / 0.1)
      SET snfSteps <- ROUND((snf - 8.5) / 0.1)
      SET rate <- 30.00 + fatSteps * 0.40 + snfSteps * 0.30
      SET amount <- ROUND(litres * rate, 2)
      SET total <- total + amount
    ENDIF
  ENDFOR
  DISPLAY total
END
```

Hand trace (three entries)

Entry	litres, fat, snf	fatSteps	snfSteps	rate	amount	total
1	10, 3.5, 8.5	0	0	30.00	300.00	300.00
2	8.5, 4.2, 8.7	7	2	33.40	283.90	583.90
3	12, 3.1, 8.3	-4	-2	27.80	333.60	917.50

The program output matches this trace (917.50).

Alternative concept and complexity: Storing every entry in a list and adding at the end uses O(n) time and O(n) space. A running total uses O(n) time but only O(1) space. Storing is still needed when we must print a statement of all entries, sort/search them, find maximum/minimum, or re-calculate after a rate correction.

Task 9 – Implement and Test the Calculator

```
from milkrate import rate_per_litre, payout


def parse_number(text):
    """Returns float or None if text is not a valid number."""
    text = text.strip()
    if text.replace(".", "", 1).isdigit():
        return float(text)
    return None


def check_entry(litres, fat, snf):
    """Returns " if the entry is valid, otherwise the error message."""
    if litres is None or fat is None or snf is None:
        return "not a number"
    if litres <= 0:
        return "litres must be positive"
    if not 2.0 <= fat <= 7.0:
        return "fat must be 2.0 to 7.0"
    if not 4.0 <= snf <= 12.0:
        return "snf must be 4.0 to 12.0"
    return ""


def ten_day_total(entries):
    """Sum of payouts for a list of (litres, fat, snf)."""
    return round(sum(payout(l, f, s) for l, f, s in entries), 2)


def run_tests():
    assert payout(10, 3.5, 8.5) == 300.00        # standard
    assert payout(10, 4.2, 8.7) == 334.00        # above
    assert payout(10, 3.1, 8.3) == 278.00        # below
```

```
    assert rate_per_litre(2.0, 8.5) == 24.00       # lowest fat
    assert rate_per_litre(7.0, 8.5) == 44.00       # highest fat
    assert check_entry(parse_number("abc"), 4.0, 8.5) == "not a number"
    assert check_entry(-5, 4.0, 8.5) != ""
    assert check_entry(10, 7.5, 8.5) != ""
    assert check_entry(10, 1.9, 8.5) != ""
    five = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3), (10, 2.0, 8.5), (10, 7.0, 8.5)]
    assert ten_day_total(five) == 1597.50
    print("All tests passed")

if __name__ == "__main__":
    run_tests()
```

Output: All tests passed

Test table (expected vs actual – actual values from the program)

#	Case	Input (L, fat, snf)	Expected	Actual	Pass
1	Standard (boundary 3.5 / 8.5)	10, 3.5, 8.5	rate 30.00, pay 300.00	30.00, 300.00	Yes
2	Fat and SNF above	10, 4.2, 8.7	33.40, 334.00	33.40, 334.00	Yes
3	Fat and SNF below	10, 3.1, 8.3	27.80, 278.00	27.80, 278.00	Yes
4	Lowest fat	10, 2.0, 8.5	24.00, 240.00	24.00, 240.00	Yes
5	Highest fat	10, 7.0, 8.5	44.00, 440.00	44.00, 440.00	Yes
6	SNF above only	10, 3.5, 9.0	31.50, 315.00	31.50, 315.00	Yes
7	SNF below only	10, 3.5, 8.0	28.50, 285.00	28.50, 285.00	Yes
8	Text	abc	rejected	not a number	Yes
9	Negative quantity	-5, 4.0, 8.5	rejected	litres must be positive	Yes
10	Fat too high	10, 7.5, 8.5	rejected	fat must be 2.0 to 7.0	Yes
11	Fat too low	10, 1.9, 8.5	rejected	fat must be 2.0 to 7.0	Yes

Hand-calculated set of 5 entries

Entry	Hand calculation	Amount
10 L, 3.5, 8.5	10 × 30.00	300.00
8.5 L, 4.2, 8.7	8.5 × (30 + 7×0.4 + 2×0.3) = 8.5 × 33.40	283.90
12 L, 3.1, 8.3	12 × (30 − 4×0.4 − 2×0.3) = 12 × 27.80	333.60
10 L, 2.0, 8.5	10 × (30 − 15×0.4) = 10 × 24	240.00
10 L, 7.0, 8.5	10 × (30 + 35×0.4) = 10 × 44	440.00
Total		1597.50 (program: 1597.5, assert passed)

Alternative concept and complexity: Typing inputs by hand every time is slow, you must remember the expected answers, and it is easy to skip cases. An assert-based test function runs all cases in a fraction of a second and reports failure automatically. After every change the manual method takes minutes (time grows with number of tests), the automatic method takes seconds and stays almost constant.

Task 10 – Compare Expectations with Execution

Method: random but repeatable entries (seed 1), litres 1–60, fat 2.0–7.0, snf 7.5–9.5; time measured with time.perf_counter(). Expected: float and Decimal agree to the paise, Decimal is slower. Observed (real):

Payments	float total	Decimal total	Difference	float time	Decimal time
300	328840.66000000015	328840.66	1.5×10^{-10}	0.0002 s	0.0006 s
6,000	6219373.350000008	6219373.35	8×10^{-9}	0.0038 s	0.0145 s
150,000	155524456.2700015	155524456.27	1.5×10^{-6}	0.093 s	0.349 s

- When rounded to 2 decimals the float and Decimal totals agree in all three cases, but the raw float error grows as the number of payments grows (from 10^{-10} to 10^{-6}). Decimal is about 3.5–4 times slower, still only 0.35 s for 150,000 payments.
- Rounding each payment vs rounding only the total: with 1-decimal litres and these rates every amount already has 2 decimals, so both gave the same total (gap 0.00). With two-decimal litres they differ: 7.35 L × 33.45 = 245.8575. Rounding each gives 245.86 × 300 = 73,758.00, rounding the total gives 73,757.25 – a Rs 0.75 difference. The bank pays each farmer a

rounded amount, so the correct bank amount is the sum of rounded payments.
Alternative concept and complexity: Rate table (dictionary): for 150,000 entries with only 1,071 different (fat, snf) pairs, recalculating took 0.088 s while the table took 0.042 s (about 2× faster). Time per entry is O(1) lookup versus O(1) calculation (but with a bigger constant); extra space is O(k) – the dictionary for k = 1,071 pairs took about 37 KB. Worth it when many entries repeat the same pairs.

Task 11 – Recommend and Defend the Final Design

- **Recommendation –** Types: Decimal (built from strings) for money, litres, fat and SNF; int for step counts and entries; str for IDs. Experiments showed floats drift (Rs 33400.30000000061, error grows with n) and Decimal(float) copies the error.
- Rounding rule: round every payment to paise (ROUND_HALF_UP, not Python's half-even), then add. This is what the bank actually pays; Task 10 showed rounding only the total can differ by rupees.
- Structure: module milkrate with named constants and small functions (rate_per_litre, payout, ten_day_total), tested with asserts.
- Rate revision: change the constant(s) at the top of the module (or the rate card file) once, run the assert tests, and distribute the new module. No other code changes.
- Limitations: only fat and SNF in fixed ranges; no database or file storage; one rate card at a time; Decimal is about 4× slower than float.
- District-union scale: for 150,000 payments Decimal took 0.35 s, so the design remains correct and fast enough; for millions of records it would still be linear O(n) and accurate, though storage would need a database.

Alternative concept and complexity: Reading the rate card from a JSON file at start: loading three values takes about 1.5 µs (json.loads) plus file-open time of milliseconds – negligible, and it occupies only a few hundred bytes. The benefit is big: a rate revision means editing a text file; no program change or redistribution. The risk is a bad or missing file, so we must validate the values when loading. The recommended final design keeps constants in the module but can load them from JSON when the union revises rates often.

Task 12 – Re-evaluate and Adapt the Design

Decision: MODIFY the original design (keep the module structure, change the maths and add season and bonus).

A. Impact analysis

- What changed: readings now have 2 decimals (e.g. 4.25); base rate and fat step depend on the season (Apr–Jun: Rs 32.00 and Rs 0.45); a bonus of Rs 0.50 per litre applies when a farmer supplies more than 500 L in ten days.
- Constants affected: BASE_RATE and FAT_STEP_VALUE are no longer single values – they become a rate card per season. New constants: BONUS_LIMIT = 500, BONUS_RATE = 0.50.

SNF_STEP_VALUE (0.30) is unchanged.

- Functions affected: rate_per_litre needs a season; ten_day_total needs the bonus; a new bonus() function is added. payout stays almost the same.
- Two-decimal readings: round((fat-3.5)/0.1) rounds to a whole 0.1 step and ignores the second decimal; 4.21, 4.24 and 4.26 would be rounded to 7, 7, 8 steps. The step count must now be fractional (4.25 → 7.5 steps) and calculated with Decimal.

B. Evaluation of the original design (worked examples)

Reading (snf 8.5)	Original design rate	Correct (new design)	Problem
fat 4.21	32.80	32.84	Under-pays Rs 0.04/L
fat 4.24	32.80	32.96	Same price as 4.21 – Rs 0.16/L under
fat 4.26	33.20	33.04	Over-pays Rs 0.16/L
fat 4.25, snf 8.65	33.80	33.45	Over-pays Rs 0.35/L: 7.5 → 8 and 1.5000000000000036 → 2

Verdict: the module structure, named constants and function split remain appropriate (retain), but the single rate card, the round()-based steps and float arithmetic are not (modify).

C. Adaptation – working code (milkrate2.py)

```
"""Rate card v2: two-decimal readings, seasonal rates and ten-day bonus."""
from decimal import Decimal as D, ROUND_HALF_UP

PAISE = D("0.01")
STD_FAT, STD_SNF, STEP = D("3.5"), D("8.5"), D("0.1")
RATE_CARDS = {
    "normal": {"base": D("30.00"), "fat": D("0.40"), "snf": D("0.30")},
    "summer": {"base": D("32.00"), "fat": D("0.45"), "snf": D("0.30")},   # April-June
}
SUMMER_MONTHS = (4, 5, 6)
BONUS_LIMIT = D("500")      # litres in ten days
BONUS_RATE = D("0.50")      # Rs per litre

def money(x):
    return x.quantize(PAISE, rounding=ROUND_HALF_UP)

def season_for(month):
    """Returns the rate-card name for a month number (1-12)."""
    return "summer" if month in SUMMER_MONTHS else "normal"

def rate_per_litre(fat, snf, season="normal"):
    """Price per litre; fat/snf are strings or Decimals (2 decimals allowed)."""
    card = RATE_CARDS[season]
```

```
    fat_steps = (D(str(fat)) - STD_FAT) / STEP     # exact, may be fractional
    snf_steps = (D(str(snf)) - STD_SNF) / STEP
    return money(card["base"] + fat_steps * card["fat"] + snf_steps * card["snf"])

def payout(litres, fat, snf, season="normal"):
    """Amount for one delivery, rounded to paise."""
    return money(D(str(litres)) * rate_per_litre(fat, snf, season))

def bonus(total_litres):
    """Bonus for the ten days: Rs 0.50 on every litre if total > 500 L."""
    t = D(str(total_litres))
    return money(t * BONUS_RATE) if t > BONUS_LIMIT else D("0.00")

def ten_day_total(entries, season="normal"):
    """entries: list of (litres, fat, snf). Returns (litres, payments, bonus, total)."""
    litres = sum((D(str(l)) for l, _, _ in entries), D(0))
    pay = sum((payout(l, f, s, season) for l, f, s in entries), D(0))
    b = bonus(litres)
    return litres, pay, b, pay + b
```

D. Evidence-based defence (real test output)

```
normal ['30.00', '33.40', '27.80'] ['300.00', '283.90', '333.60']
summer ['32.00', '35.75', '29.60'] ['320.00', '303.88', '355.20']
bonus 500 -> 0.00 | bonus 500.01 -> 250.01 | bonus 620 -> 310.00
500.0 L normal: litres 500, payments 15000.00, bonus 0.00,   total 15000.00
500.5 L normal: litres 500.5, payments 15015.00, bonus 250.25, total 15265.25
500.5 L summer: litres 500.5, payments 16016.00, bonus 250.25, total 16266.25
all asserts passed
```

Test	Expected	Actual
Standard milk, normal season	30.00	30.00
Standard milk, summer	32.00	32.00
4.20 fat / 8.70 snf, summer	32 + 7×0.45 + 2×0.30 = 35.75	35.75
4.25 fat / 8.65 snf, summer	32 + 7.5×0.45 + 1.5×0.30 = 35.825 → 35.83	35.83
Exactly 500 L	no bonus (not more than 500)	0.00
500.5 L	bonus 0.50 × 500.5 = 250.25	250.25

- Trade-offs: Decimal is slower (about 4×) but exact, which matters for money; the season is a parameter so old months can still be recalculated; the bonus is a separate function so its rule can change alone.

- Assumptions: the bonus applies to all litres of that farmer once the ten-day total exceeds 500 L; the season is chosen from the month of the period; payment per 0.01% is proportional to the step value (pro-rata).
- Conclusion: modifying the original design is better than a full redesign – the rules changed, not the structure – and all tests for both seasons and the bonus pass.